A guided tour of Scale

Status: a tour, not the specification. This document introduces the whole language to an experienced programmer who has not met concatenative programming, append-only images, or set-theoretic typing before. The normative rules are in `docs/specification.typ`; this page is the mental model.

The one-sentence version

Scale is a **set-theoretic, concatenative language** whose durable values live in an **append-only image** and whose execution state is never serialized. One semantic core has three front ends: the core concatenative surface, the Prose sentence grammar, and the prose program surface. They all compile to the same stack machine; none has a private evaluator.

That sentence contains three ideas that deserve unpacking:

- **concatenative**: programs are sequences of words; words transform a stack;
- **set-theoretic**: values and types are built from sets, with an optional name attached;
- **image**: data lives in one append-only store, and a running program is a transient view over that store.

User-facing and implementation-wise

This tour separates two kinds of information:

- **User-facing**: the syntax and semantics you write, the values and effects you observe, and the language's distinctive ideas. If you are here to use Scale, this is the part that matters.
- **Implementation-wise**: how the runtime represents, validates, lowers, or ships those ideas. It becomes interesting when a design choice is unusual or explains a visible guarantee.

Implementation-wise material appears in orange **Sidenote** boxes. They are optional reading. A user can skip every one and still have the complete language surface. The boxes exist to give the practical material contrast, not to turn this into an internals manual.

> **Sidenote — implementation-wise.** The implementation has several internal representations: a symbolic term list, a canonical portable JSON document, and a lowered instruction stream. They are compiler and runtime forms, not additional user surfaces. The surface you write is the language; the representations are how one implementation carries it.

Why the design looks like this

The project is trying to make a small, composable core carry a rich surface. Words compose by juxtaposition. A quotation lets code travel as data. The image gives identity, sharing, persistence, and a natural place for definitions. Sets give one representation for values, types, and dispatch. Actors and host words put effects at a boundary instead of in the middle of the evaluator. The Prose front ends let a child or a domain author write sentences while the same core runs underneath.

This is deliberately not a second language per front end. A sentence, a book definition, and a direct stack program eventually become the same units and instructions.

First mental model: the stack

If you know an ordinary expression language, start by forgetting named arguments. A Scale program is a sequence of tokens. A token that is not syntax is a **word call**. A word pops its operands from a stack and pushes its results. The last value pushed is the top of the stack.

```
2 3 add
```

Read it left to right:

After	Stack, bottom to top
`2`	`2`
`2 3`	`2; 3`
`2 3 add`	`5`

There is no hidden argument order. `2 3 add` and `3 2 add` differ only in which value is which, and for subtraction the difference matters:

```
10 3 subtract     \ 7
3 10 subtract     \ -7
```

The core arithmetic words are `add`, `subtract`, `multiply`, `divide`, `lessThan`, `greaterThan`, and `equals`. Comparisons produce a scalar: zero is false, non-zero is true.

Words compose by juxtaposition

Composition is concatenation. To compute `(2 + 3) * 4`:

```
2 3 add 4 multiply
```

Each word leaves its result where the next word expects it. This is the whole execution model. There is no precedence table for words and no expression grammar to learn for the core; the sequence is the tree.

The stack words are the usual concatenative plumbing:

Word	Effect
`duplicate`	`a` becomes `a a`
`drop`	`a` becomes nothing
`swap`	`a b` becomes `b a`
`over`	`a b` becomes `a b a`
`rotate`	rotates three values

For example:

```
5 duplicate add          \ 10
2 3 swap subtract        \ 1
```

Sidenote — implementation-wise. The interpreter is a token loop over a term list and an explicit stack. A word is a host function from `EvaluationState` to `Result<EvaluationState, EvaluationError>`. The compiled path lowers the same program to a small instruction stream. The stack is the user model; the term list and the instruction stream are impossible to see from the surface unless you ask for them.

Quotations: code as data

`{ ... }` collects the tokens up to the matching `}` into a **quotation** and pushes its address. A quotation is data. It does not run until something applies it.

```
5 { duplicate } apply add     \ 10
```

`apply` pops a quotation or a closure and runs it. A bare quotation runs in the current frame; a closure runs in the frame it captured and then restores the caller's frame.

`if` is a quotation consumer:

```
condition { then } { else } if
```

The condition must be a scalar. Zero chooses the else branch; non-zero chooses the then branch. The chosen quotation is spliced into the program and runs on the same stack.

This is the key difference from a conventional language: control flow does not need a second evaluator. A branch is a word that selects code, and code is a value.

Defining words

The ordinary definition word is `logic`:

```
logic inc { 1 add }
5 inc                    \ 6
```

`logic` reads the name and the quotation body, then stores the body under the name in the user-word table. A later definition under the same name wins. Calling `inc` later runs the stored body. The name is not in scope in a `logic` body, so a self-call is refused.

`cycle` is the recursive form. The body uses the plain name:

```
cycle fact {
  duplicate 0 equals
  { drop 1 }
  { duplicate 1 subtract fact multiply }
  if
}
5 fact                   \ 120
```

Mutual recursion uses `and`, F#-style, and every group name is in scope in every body:

```
cycle even { odd }
and odd { even }
```

The older stack form `body "name" define` remains available for now. There is no `recurse` word.

Local bindings use `let` and closures:

```
42 "x" let
x                        \ 42
```

`let` pops the value and the name, creates a child frame with that binding, and makes it current. `lookup` walks the frame chain and then the global binding table. A bare name resolves lexically before the global word table, so a local binding shadows a built-in word.

A quotation bound with `let` is still data; apply it to run it:

```
{ 1 add } "inc" let
5 inc apply              \ 6
```

`lambda` is the explicit closure constructor: `{ body } lambda` captures the current frame and pushes a closure. `apply` runs it later.

Failures are values

In the interpreter, a failure is a value, not a thrown exception. A word returns either a new state or a named error; the main loop attaches source position and returns a diagnostic. A named reason can be matched. An untyped string refusal is not part of the language.

That matters for the surface design: an unresolved word, a stack underflow, a type mismatch, a missing book, and a refused marker are all reported by name. The system does not turn a mistake into a silent empty result.

The Prose sentence surface

The Prose sentence grammar is a controlled English for statements and transitions. A statement has a quantifier, a subject, optional conditions, and a verb:

```
Every player that is alive can jump.
```

`every` is universal affirmative, `no` is universal negative, and `some` is existential. A transition has a trigger and an effect:

```
When player is alive, every player can jump.
```

The grammar is order-sensitive: the quantifier is sentence-initial, `that` introduces conditions, and the sentence ends with a full stop. This front end is used for specification and verification, not as a general expression language.

The prose program surface is the sentence-shaped general language. A program is a sequence of sentences ending in full stops. A match can be inline:

```
match 5, when 1 then "one", when 2 then "two", otherwise "many".
```

or written one clause per line:

```
match 5.
when 1 then "one".
when 2 then "two".
otherwise "many".
```

There is no `end` keyword. A line break does not start a new sentence, so the clause words stay lowercase. A nested match is closed with `close`. Names are lowercase words separated by spaces and canonicalized through the vocabulary; quoted text keeps its case.

Shell effects

`run` and `capture` are host words, not evaluator special cases:

```
run "echo" with "hello".
capture "echo" with "hello".
```

`run` forwards standard output and standard error and pushes the exit status. `capture` forwards standard error and pushes standard output as text. Both record the last command status. A non-zero status is a value, not an immediate abort. The current shell slice executes one command sentence at a time; pipelines, redirection, variables, and job control are later stages.

Books and spells

A `.book` file is a module of definitions. A book with a `main` definition is a **spell** (a CLI tool); without `main` it is a **book** (a library).

```
logic double x = multiply(x 2).
logic quad x = double(double(x)).
```

A book is one recursive group. All definitions are available to every body, and `cycle` makes recursion explicit:

```
cycle fact n = match n, when 0 then 1, otherwise multiply(n fact(subtract(n 1))).
```

The markers are `logic` for an ordinary definition, `cycle` for a recursive definition, and `and` for a cycle group, F#-style. A `logic` body may not call its own name; a `cycle` body may call any name in its group. The marker is a diagnostic, not a second semantics. There is no `recurse` word.

From prose, a book is called with `cast`:

```
cast("math.book" "double" 21).
```

From the CLI, `--book` runs a spell and `--cast` calls an exported word. A book compiles to a package document, and its definitions become units in one group with an explicit initialization order and classification.

> **Sidenote — implementation-wise.** A book compiles to a package document and then to an image: each definition becomes a unit with explicit arity and capture slots, and all definitions form one group with an `InitOrder` and a binding classification. The book runner executes the requested unit through the group runtime. The image schema and the group metadata are how the runtime preserves the recursion guarantee; the writer only sees definitions and calls.

Sets and sorts

Scale uses sets for two jobs that most languages split between values and types. The short version is:

- a **set** is a collection with membership, subset, union, intersection, and complement;
- a **sort** is a set with an optional name;
- structural equality reads the members; nominal equality reads the name.

If you have not used set theory as a programming model, this is enough:

Idea	Meaning
membership	`x ∈ S`: `x` is in `S`
subset	`A ⊆ B`: every member of `A` is in `B`
union	`A ∪ B`: members of either
intersection	`A ∩ B`: members of both
complement	`U \ A`: members of the universe not in `A`
extensionality	two sets with the same members are the same set

`[ ... ]` is an extension literal. The body is evaluated and its results are admitted to a set:

```
[ 1 2 3 ]           \ the set {1, 2, 3}
[]                  \ the empty set
```

`2 :small is` tests membership after `small` has been declared or sealed. `model` declares a domain; `admit` extends it. `is` is membership, and sets deduplicate: a set has no multiplicity and no order.

Sorts: a set with a name

A sort is the pair `(N, X)`: an optional name `N` and an extension `X`. Two sorts can be compared in two ways:

- **structural**: compare their extensions; this is extensionality;
- **nominal**: compare their names.

Sealing attaches a name:

```
[ ] :player seal
[ 1 2 3 ] :player seal
```

`S :Name seal` makes equality name-based. `unname` drops the name and returns to structural equality. This is the dial between "two structurally identical sets are the same type" and "two sets with different names are different types, even if their members coincide."

The set words are the operations you would expect: `set-union`, `set-intersect`, `set-except`, `set-member`, `set-complement`, `set-subset`, `merge`, `filter`, `sort-of`, `same-shape`, and `same-name`. `and`, `or`, and `nor` are the lattice operations. `set-subset` is the value-level subset operation; a type-level subtype relation is not a word until a consumer needs it.

Why do this? Because one representation can describe values, types, dispatch, and refinements without a parallel hierarchy. A sort can be a finite set, a named set, a predicate over a base set, or a constructor. Dispatch can select the most specific sort. A refinement can narrow a set. Verification can encode the same fields as sorts.

Sidenote — implementation-wise. A sort is represented as a pair `(Name, Extension)`; set operations are image-resident association vectors with latest-write-wins lookup. `seal` adds the nominal half, and `unname` removes it. The practical consequence is the whole point: one record can describe a finite set, a named type, a predicate sort, or a constructor, and dispatch can compare them without a separate type hierarchy.

Patterns, classes, and dispatch

Pattern matching is the surface for inspecting a value:

```
subject { pattern -> body | pattern -> body } match
```

Clauses are tried in order. A matching clause binds variables and splices its body. If no clause matches, the result is a named `NoClauseMatched`, not a silent zero. Patterns support fresh variables, pins, repeated variables, shadowing, constructors, typed patterns, and guards. The lowered subset is smaller than the interpreter's set; unsupported forms are named refusals.

Classes and instances are image-resident records. A class has a name, a superclass chain, and slots; an instance names its class and holds slot values. Generic functions and methods provide multiple dispatch: the most specific applicable method wins, an ambiguous set is a named failure, and no applicable method is also named.

Sidenote — implementation-wise. The IR is implementation-wise. A program has a symbolic term list, a canonical portable JSON document, and a lowered instruction stream; none is a second language. Pattern matching is the place where the internal design becomes interesting: it has a dedicated pattern IR because patterns need a closed node universe, structural analysis, a portable document, and a lowering path. That is why pattern syntax can be checked before it runs. You can use pattern matching without reading any of that machinery.

Actors, channels, and supervision

Actors are the concurrency model. An actor has a status, a parent, a private handle, a mailbox, links, monitors, and a pending exit.

- `spawn` creates a child;
- `self` pushes the current handle;

- `yield` gives the CPU to the next ready actor;
- `halt` ends normally;
- `kill` ends with a reason.

A handle is a capability, not an integer: a program cannot forge one by guessing a number. Messages go to mailboxes. A blocked receive is a distinct outcome, not an `Error`. Links deliver abnormal exits; monitors observe an exit once; supervision restarts children within a policy.

Channels and ChannelEvents are the lower-level communication surface. The scheduler is deterministic for a fixed program and a fixed order of external events; it is not a thread pool hiding preemption.

Effects and ownership

Scale keeps effects at boundaries. A host word is offered to an attached adapter; the Prelude does not get OS authority directly. The shell host is the current example: `run` and `capture` are host words, and a script's exit status comes from the last command.

Ownership is a type-level usage mode:

- `ExactlyOnce` for a linear resource;
- `AtMostOnce` for an affine reference;
- `AnyNumberOfTimes` for an immutable image value.

`linearity` declares the mode; `own`, `borrow`, `consume`, and `release` are the operations. Ownership state is execution state, not image data.

Sidenote — implementation-wise. Actors are image-resident records; the scheduler, ownership maps, waiters, and mailboxes are execution state. A handle is a capability entry, not the actor's address. Host words cross an ABI callback table, so the Prelude does not hold OS authority directly. These boundaries are why effects are testable and why an actor restart does not silently reuse a stale handle.

Verification

The verifier is Z3, in-process. A property becomes a typed predicate over the field sorts. The result is one of `Proved`, `Falsified` with a model, `Unknown`, or `Unavailable`. `Unknown` is reported, never treated as a proof; a resource limit is `Unavailable`, not a pass.

The refinement loop takes a failed property, a candidate guard, and an obligation, and selects a guard that closes the failure. Decision tables are the partition form: completeness is a union covering the domain, and disjointness is an empty pairwise intersection. The same set vocabulary describes the table rows, the lattice, and the SMT fields.

Sidenote — implementation-wise. The solver is Z3, in-process (`Microsoft.Z3` 4.12.2, with `/usr/bin/z3` as the offline fallback). An obligation is a typed predicate over field sorts; a resource limit is `Unavailable`, and an inconclusive solver result is `Unknown`, never a pass. The refinement loop selects a proved guard; it does not guess one.

The image and the runtime

The image is the durable side of the system. It holds identities, tensors, text, association tables, units, groups, kinds, predicates, and package metadata. It is append-only: an update writes a new vector and re-points a root; it never rewrites or truncates an entry.

The running program is the other side: stack, current frame, scheduler, ownership, history, handlers, and program counter. Execution state is never serialized. A reader loads the image and re-threads the named roots.

> **Sidenote — implementation-wise.** An `EntryIdentity` is a struct wrapping an integer, not a host integer in the language; it projects to a rank-0 tensor at a boundary. Text is interned in the store's text cache. The store is append-only: an update writes a new association vector and re-points a named root. Those roots are execution state, so a reader must reattach them after loading. This is why "the image is the durable side" is more than a slogan.

The current image schema is 2. A unit carries a name, source hash, dependencies, explicit arity, capture slots, and instructions. A unit group carries its units, `InitOrder`, and a binding classification. The image is validated before any unit runs.

The ABI is a single integer version, currently 2. The C ABI header is the contract. The kernel has no Prelude logic; the Prelude is the symbolic implementation and the kernel executes image-resident units. The NativeAOT kernel and the C conformance harness are the native gate.

A program has four representations: surface text, the elaborated term list, a canonical portable JSON document, and a compilation target. The lowered instruction stream is a target, not a second runtime. The portable form carries no host objects.

How the pieces fit

A useful way to read the system:

Layer	What it owns
surface	tokens, sentences, expressions, books, shell scripts
front ends	Prose sentence grammar, prose programs, books, core tokens
IR	expressions, patterns, packages, terms, lowered instructions
image	durable identities, tensors, text, definitions, groups
runtime	stack, frames, scheduler, ownership, history, handlers
kernel	lowered abstraction, image loader, term verifier, ABI
host adapters	marshalling, effects, IPC, OS authority
verification	Z3, refinement, diagnostics, decision tables

A call is not a reference. A lowered word call resolves to a unit or a word; it does not silently push a lexical value. A book call resolves through its group. A closure runs the frame it owns. That one contract is what keeps the surfaces from growing separate call semantics.

A small end-to-end example

A prose program:

```
match 5, when 1 then "one", when 2 then "two", otherwise "many".
```

A book library:

```
logic double x = multiply(x 2).
logic quad x = double(double(x)).
```

A cast:

```
cast("math.book" "quad" 21).       \ 84
```

A recursive book:

```
logic fact n = match n, when 0 then 1, otherwise multiply(n fact(subtract(n 1))).
```

A shell script:

```
run "echo" with "hello".
capture "echo" with "hello".
```

A set and a sort:

```
[ 1 2 3 ] :small seal
2 :small is
```

The point is not that every surface is the same syntax. The point is that every surface reaches the same values, the same image, and the same execution model.

What is implemented, and what is not

The repository is in alpha. The current acceptance suite is 1530 checks (1488 examples and 42 properties), with 33 granular cases. The NativeAOT kernel passes the C conformance harness.

Implemented today includes the core concatenative surface, the prose sentence grammar, the prose program surface, books and spells with automatic recursion and markers, the shell Stage 1 effects, patterns, sets and sorts, classes and dispatch, actors and channels, ownership, the image, persistence, the kernel ABI, the C adapter, and the Z3 verification path.

Not implemented, or deliberately deferred:

- the F# package classification and manifest tiers;
- the full Scale Prelude port (the F# Prelude remains the shipped reference);
- a REPL;
- shell pipelines, redirection, variables, and job control;
- full static type checking and effect rows;
- sequence and recursive types; multisets and co-products;
- cross-book imports and the full capability sandbox.

The status matrix in `docs/specification.typ` and the decision index are the authority when this page and a feature disagree.

A suggested reading path

1. Write ten core programs with the stack and quotations.
2. Write one prose program with a match and one shell script with `run`/`capture`.
3. Write one book with two functions and a recursive function.
4. Read the set/sort section of the specification and try `[ ]`, `seal`, `is`, and the set words.
5. Read the actor section and the shell effect boundary.
6. Read `docs/specification.typ` by section, then the feature document for the part you are changing.
7. Use `docs/decision-index.typ` to find one decision; do not read the decision archive end to end.

The language is small at the core and deliberately broad at the edges. Read the core first, then one edge at a time.